3D Navigation 실습 보고서

1. 실습 1: 기본 내비게이션 (AMCL + 기본 지도)

배경 이론: SLAM과 AMCL

로봇이 자율주행을 하려면 지도와 지도 위에서의 내 위치가 모두 필요하다. 이를 해결하는 대표적인 방법이 SLAM과 AMCL이다.

구분	SLAM	AMCL
정식 명칭	Simultaneous Localization And Mapping	Adaptive Monte Carlo Localization
지도	없는 상태에서 직접 생성	이미 있는 지도를 사용
역할	지도 생성과 위치 추정을 동시에 수행	위치 추정만 수행
초기 위치 지정	필요 없음	필요 (RViz의 2D Pose Estimate)

SLAM의 동작 과정은 다음과 같다.

1. 로봇이 움직이며 라이다로 주변을 스캔한다.
2. 연속된 스캔을 서로 맞추어(scan matching) 로봇의 이동량을 추정한다.
3. 추정한 위치에 스캔 결과를 덧붙여 지도를 넓혀 간다.
4. 이전에 방문한 장소로 돌아오면 이를 인식하여(Loop Closure) 누적된 오차를 보정한다.

AMCL의 동작 과정은 다음과 같다.

1. 지도 위에 로봇이 있을 수 있는 후보 위치인 파티클(particle)을 여러 개 뿌린다.
2. 각 파티클 위치에서 예상되는 라이다 스캔과 실제 스캔을 비교하여 점수를 매긴다.
3. 점수가 높은 파티클 주변에 새 파티클을 다시 뿌리고(resampling), 점수가 낮은 파티클은 제거한다.
4. 로봇이 움직일수록 파티클이 실제 위치로 모이며, 파티클 구름의 크기가 작아진다.

실습 결과

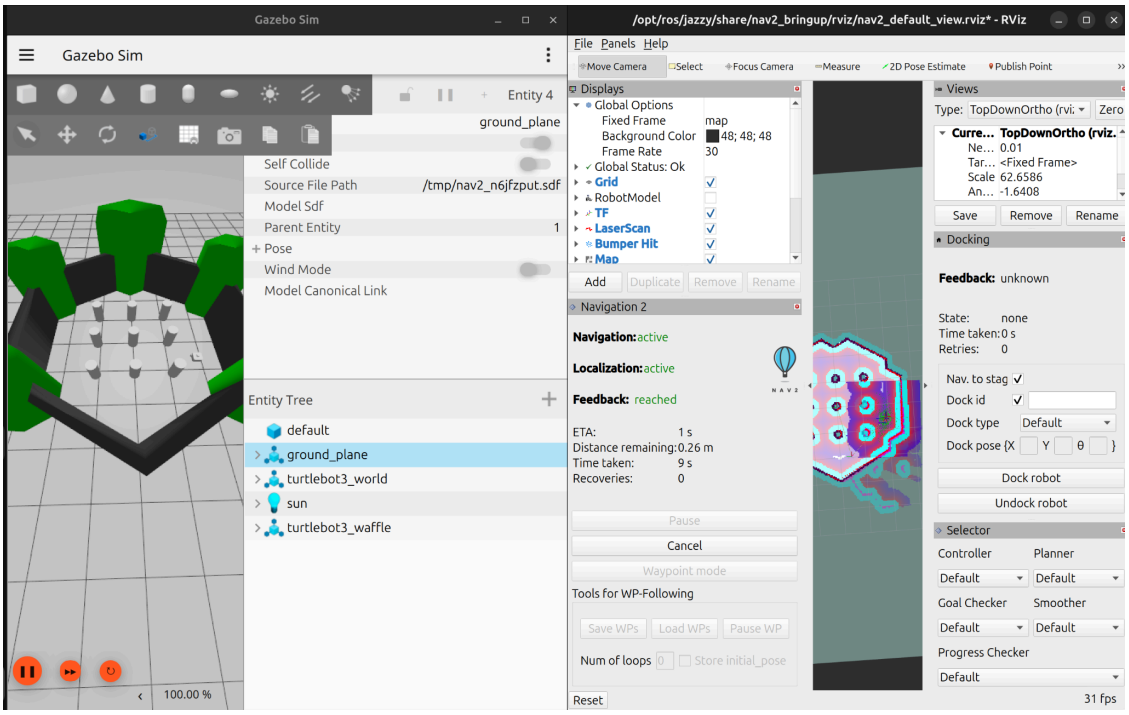


그림 1. 2D Pose Estimate로 초기 위치를 지정한 후의 RViz 화면

그림 1에서 바깥 벽과 가운데 기둥(장애물)의 표면에 빨간색 점이 표시된 것을 확인할 수 있다. 이 빨간 점은 로봇의 라이다가 측정한 스캔 데이터이며, 스캔이 지도의 벽 위치와 정확히 겹친다는 것은 AMCL이 로봇의 위치를 올바르게 추정하고 있음을 의미한다. 또한 로봇 주변에 초록색 파티클이 모여 있는 것을 볼 수 있는데, 이는 AMCL이 추정한 로봇의 후보 위치들이 실제 위치 근처로 수렴하였음을 나타낸다.

2. 실습 2: SLAM (지도 만들기)

실습 과정

SLAM을 실행한 뒤, 새 터미널에서 teleop_twist_keyboard를 실행하여 키보드로 로봇을 조종하였다.

```
source /opt/ros/jazzy/setup.bash && source ~/nav2_edu_ws/scripts/edu_env.sh
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

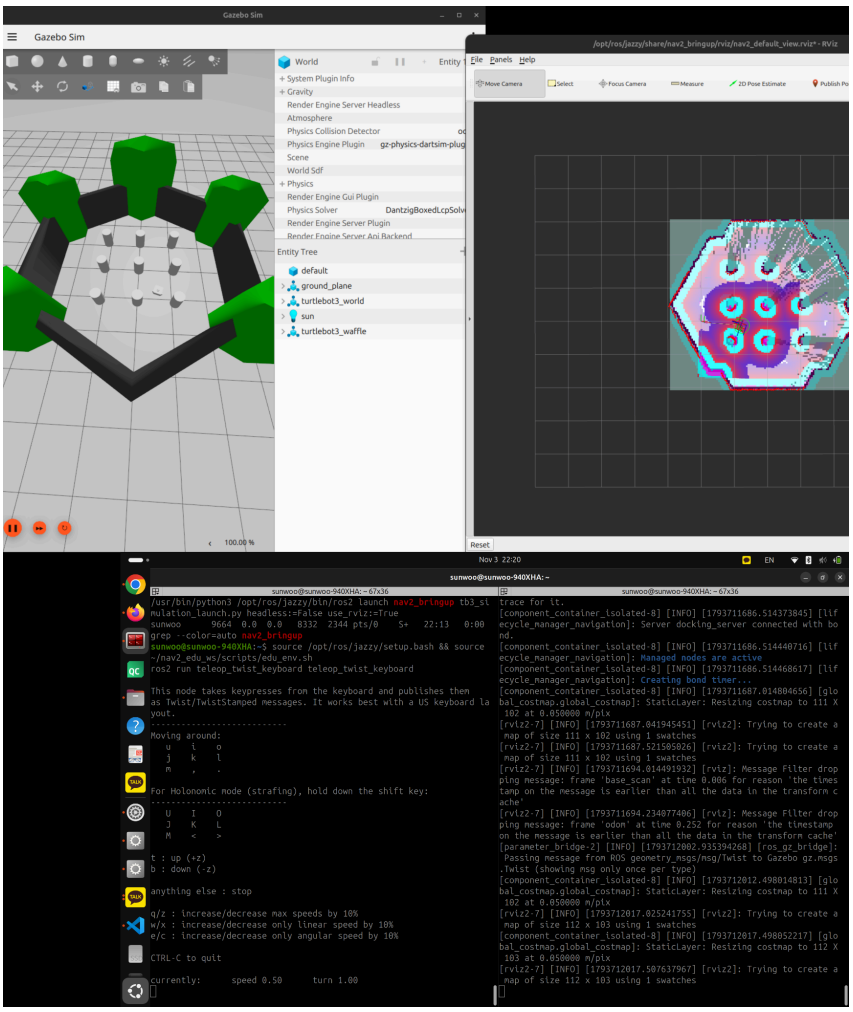


그림 2. 로봇을 조종하며 지도를 생성하는 과정

생성되는 지도의 각 색은 다음과 같은 의미를 가진다.

색	의미
흰색	빈 공간 (이동 가능)
검정	장애물 (벽)
회색	아직 탐색하지 않은 영역

주행을 시작할 때는 지도의 대부분이 회색이었으나, 로봇이 새로운 영역으로 이동할수록 회색이 사라지고 흰색(빈 공간)과 검정(벽)으로 채워지는 것을 확인하였다.

지도 저장

SLAM을 실행한 상태로 두고, 새 터미널에서 map_saver_cli를 사용하여 지도를 저장하였다.

```
source /opt/ros/jazzy/setup.bash && source ~/nav2_edu_ws/scripts/edu_env.sh
ros2 run nav2_map_server map_saver_cli -f ~/nav2_edu_ws/maps/my_map \
```

```
    --ros-args -p save_map_timeout:=10.0
```

저장 결과 my_map.pgm(지도 이미지)과 my_map.yaml(해상도·원점 정보) 두 파일이 생성되었다. 처음에는 기본 대기 시간(2초) 안에 지도를 받지 못해 "Failed to spin map subscription" 오류가 발생하였으나, save_map_timeout 값을 10초로 늘려 해결하였다.

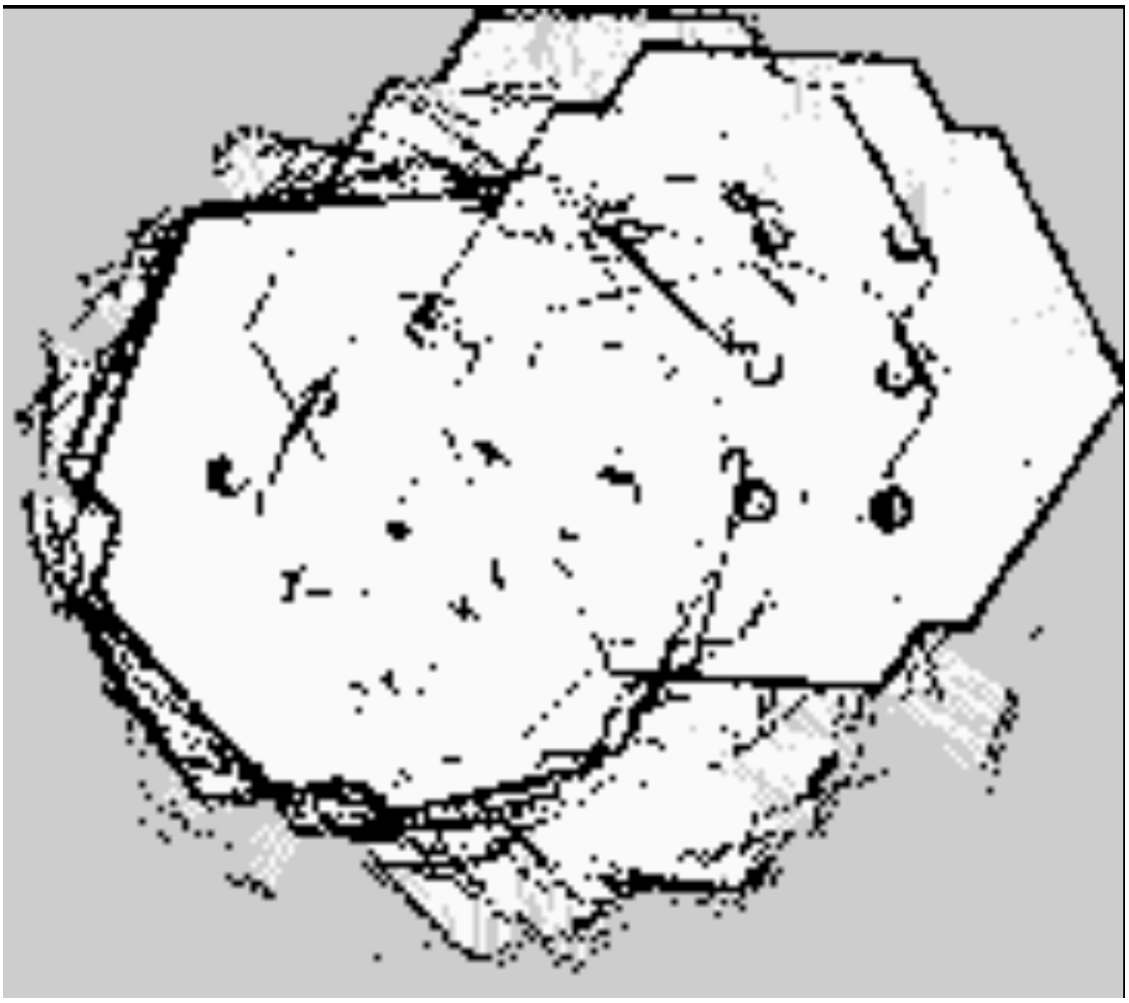


그림 3. 처음 생성하여 저장한 지도

고찰

그림 3은 처음 만든 지도로, 일부 벽이 겹치거나 어긋나게 그려졌다. 이는 조종 중에 로봇이 벽에 여러 번 부딪혔기 때문이다.

로봇이 벽에 막힌 상태에서도 바퀴는 계속 회전하므로, 바퀴 엔코더(오도메트리)는 로봇이 앞으로 이동했다고 계산한다. 그러나 실제 로봇은 제자리에 있기 때문에, SLAM이 추정한 위치와 실제 위치 사이에 오차가 생기고 지도가 어긋나게 그려진다. 이는 터틀봇 프로젝트에서 엔코더 값이 실제 이동량과 정확히 일치하지 않아 발생했던 문제와 비슷한 원리이다.

따라서 깨끗한 지도를 얻으려면 다음과 같이 주행하는 것이 좋다.

- 벽에 부딪히지 않도록 천천히 주행한다.
- 제자리에서 빠르게 회전하지 않는다.
- 이미 지나온 장소를 다시 지나가 Loop Closure가 일어나도록 하여 누적 오차를 보정한다.

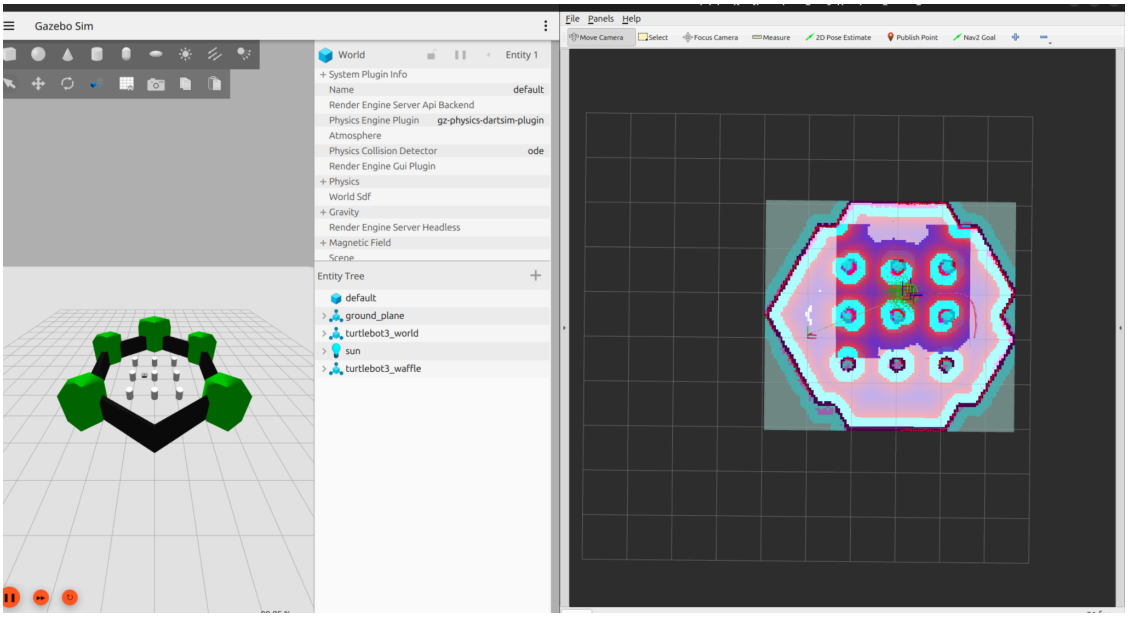


그림 4. 지도를 다시 만들어 실행하는 모습

주행 방법을 개선하여 지도를 최대한 깨끗하게 다시 만들고, 만들어진 지도 위에서 로봇을 주행해 보았다. 로봇은 문제없이 목표 지점까지 이동하였다.

3. 실습 3: 파라미터 튜닝

배경 이론

(1) Costmap과 팽창 반경

Nav2는 지도를 그대로 사용하지 않고, 장애물 주변에 가까이 갈수록 위험하다는 비용을 칠한 비용 지도로 변환하여 사용한다. inflation_radius는 이 비용을 장애물로부터 몇 m까지 칠할지 결정하는 값이다. 반경이 작으면 로봇이 장애물에 가깝게 지나가고, 반경이 크면 장애물에서 멀리 떨어져 지나간다.

구분	전역 costmap (global)	지역 costmap (local)
범위	지도 전체	로봇 주변 일정 영역

구분	전역 costmap (global)	지역 costmap (local)
용도	Planner가 전체 경로를 계산할 때 사용	Controller가 실시간으로 장애물을 피할 때 사용

(2) Planner와 Controller

Nav2의 주행은 두 단계로 이루어진다. Planner는 출발점에서 목표 지점까지의 전역 경로(선)를 계산하고, Controller는 그 경로를 따라가도록 매 순간 로봇의 속도 명령(/cmd_vel)을 생성한다. Controller를 MPPI에서 RPP로 바꾸어 비교해보았다.

구분	MPPI (기본값)	RPP (Regulated Pure Pursuit)
동작 방식	가능한 궤적을 수천 개 샘플링·시뮬레이션하여 비용이 가장 낮은 궤적 선택	경로 위 일정 거리(lookahead) 앞의 점을 목표로 따라감
계산량	큼	작음
코너 주행	장애물을 고려하여 부드럽게 회전	경로 코너를 질러가는 경향
좁은 통로	비교적 잘 통과	충돌 검사에 걸려 멈칫할 수 있음

실습 방법

lab3 스크립트는 params/nav2_params.yaml 파일을 Nav2 설정 파일로 사용한다. 파일에서 "### [LAB3]" 주석이 붙은 위치의 값을 수정한 뒤 스크립트를 다시 실행하여 결과를 비교하였다.

```
~/nav2_edu_ws/scripts/lab3_params.sh
```

(1) 팽창 반경 변경: 실행 중인 상태에서 새 터미널로 global, local costmap의 값을 같이 변경하였다.

```
source /opt/ros/jazzy/setup.bash && source ~/nav2_edu_ws/scripts/edu_env.sh
ros2 param set /global_costmap/global_costmap inflation_layer.inflation_radius 0.3
ros2 param set /local_costmap/local_costmap inflation_layer.inflation_radius 0.3
```

(2) 컨트롤러 변경: nav2_params.yaml의 controller_server → FollowPath의 plugin 값을 RPP로 바꾸고 스크립트를 재시작하였다.

```
FollowPath:
  plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
  desired_linear_vel: 0.3
  lookahead_dist: 0.6
  use_rotate_to_heading: true
```

실습 결과

(1) 팽창 반경 비교

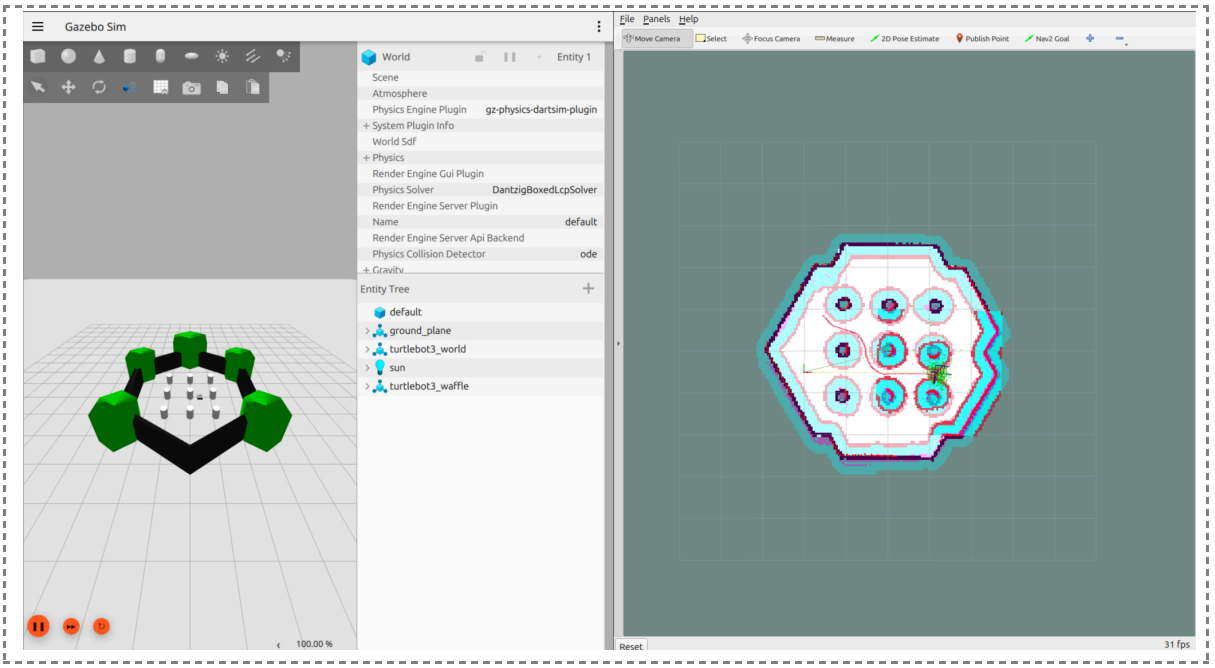


그림 5. inflation_radius = 0.3 m

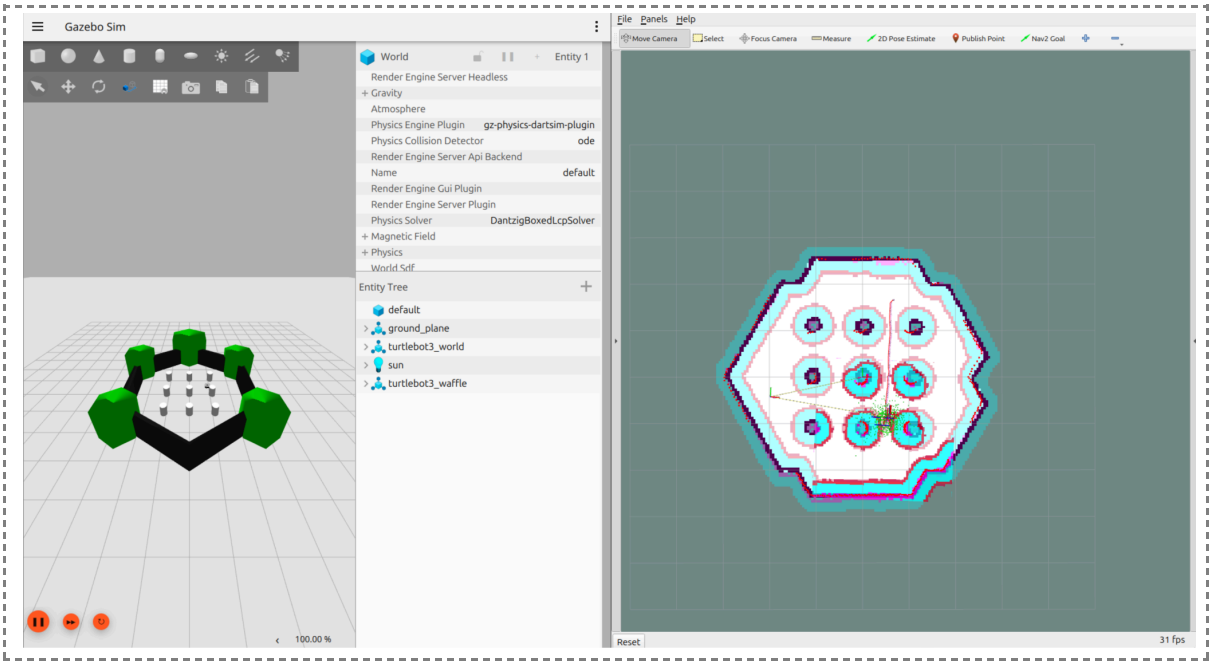


그림 6. inflation_radius = 0.7 m (기본값)

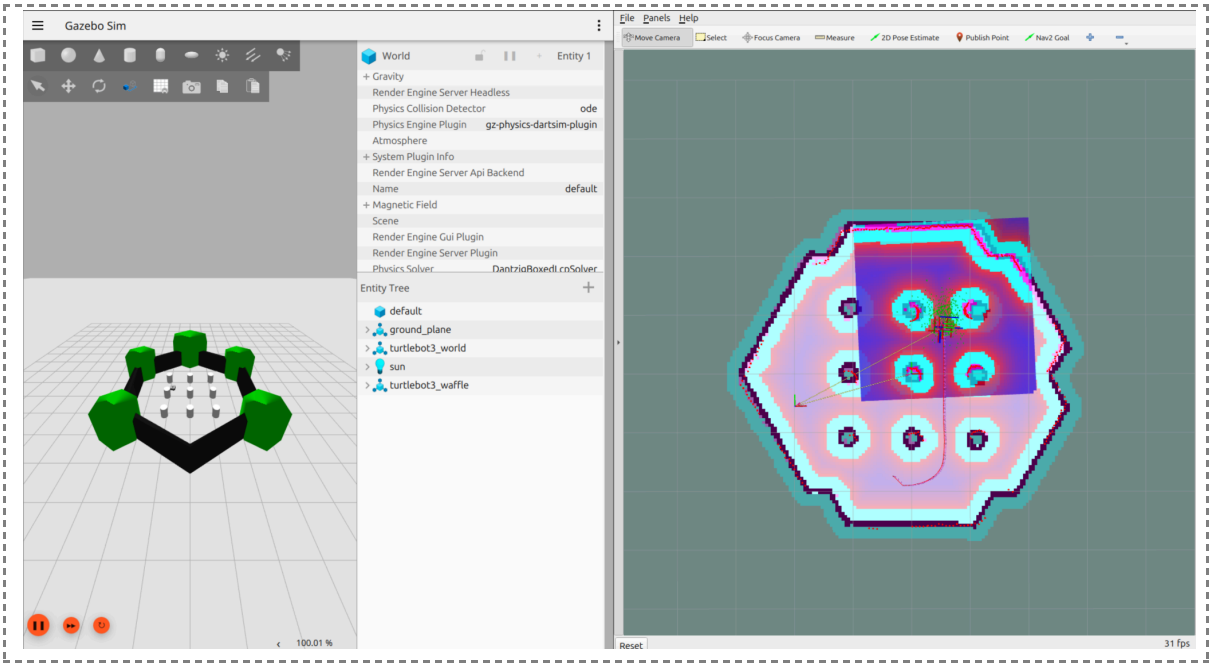


그림 7. inflation_radius = 1.0 m

inflation_radius	costmap 띠	경로 특징
0.3 m	얇음	장애물에 가깝게 붙어 지나감. 좁은 통로 통과가 쉬움
0.7 m (기본)	중간	장애물과 적당한 거리를 유지
1.0 m	두꺼움	장애물에서 멀리 떨어져 지나감.

팽창 반경이 커질수록 벽과 기둥 주변의 색칠된 영역이 두꺼워지고, 전역 경로가 장애물로부터 더 멀리 떨어져 생성되었다.

(2) 컨트롤러 비교 (MPPI → RPP)

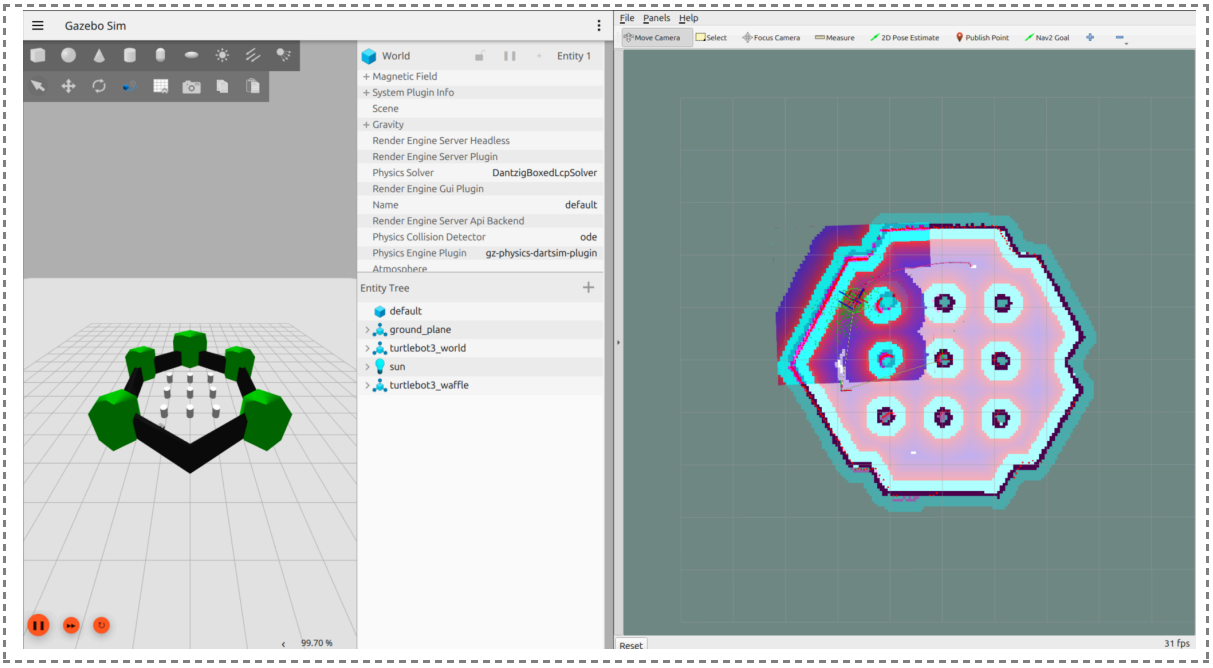


그림 8. RPP 컨트롤러로 주행하는 모습

RPP로 변경한 뒤에는 경로의 코너를 매우 인코스로 통과하였다.

고찰

- 안전과 효율의 trade-off: 팽창 반경을 작게 하면 경로가 짧아지고 좁은 곳도 통과할 수 있지만 장애물에 부딪힐 위험이 커진다. 반대로 크게 하면 안전하지만 좁은 통로를 지나가지 못한다. 따라서 로봇의 크기와 환경의 통로 폭을 고려하여 값을 정해야 한다.
- 같은 전역 경로라도 어떤 컨트롤러로 따라가느냐에 따라 실제 움직임이 달라졌다.
- lookahead_dist가 크면 부드럽게 주행하지만 코너를 더 많이 질러가고, 작으면 경로를 정확히 따라가지만 진동이 생길 수 있다.

4. 실습 4: Python으로 웨이포인트 순회 (nav2_simple_commander)

배경 이론

nav2_simple_commander는 Nav2를 Python에서 쉽게 제어할 수 있도록 만든 라이브러리이다. 내부적으로는 Nav2의 액션 서버(navigate_to_pose, follow_waypoints 등)에 요청을 보내며, 사용자는 BasicNavigator 클래스의 함수만 호출하면 된다.

함수	역할
setInitialPose()	초기 위치 설정 (RViz의 2D Pose Estimate와 동일)
waitUntilNav2Active()	AMCL과 Nav2 노드가 활성화될 때까지 대기
followWaypoints()	여러 웨이포인트를 순서대로 주행
getFeedback()	현재 몇 번째 웨이포인트로 이동 중인지 확인
isTaskComplete() / getResult()	주행 완료 여부와 결과(성공/취소/실패) 확인

실습 방법

터미널 1에서 lab1(기본 지도 + AMCL)을 실행한 뒤, 터미널 2에서 lab4 스크립트를 실행하였다.

스크립트 내부에서 ROS_DOMAIN_ID=42를 자동으로 설정하므로 edu_env.sh는 따로 불러오지 않았다.

```
# 터미널 1
~/nav2_edu_ws/scripts/lab1_basic.sh
# 터미널 2 (lab1이 다 뜬 뒤)
source /opt/ros/jazzy/setup.bash
python3 ~/nav2_edu_ws/scripts/lab4_commander.py
```

순회한 웨이포인트는 다음과 같다.

순서	좌표 (x, y)	yaw	도착 방향
초기 위치	(-2.0, -0.5)	0	+x (오른쪽)
웨이포인트 1	(0.5, -1.5)	0	+x (오른쪽)
웨이포인트 2	(1.8, 0.0)	π/2	+y (위쪽)
웨이포인트 3	(0.5, 1.8)	π	-x (왼쪽)

실습 결과

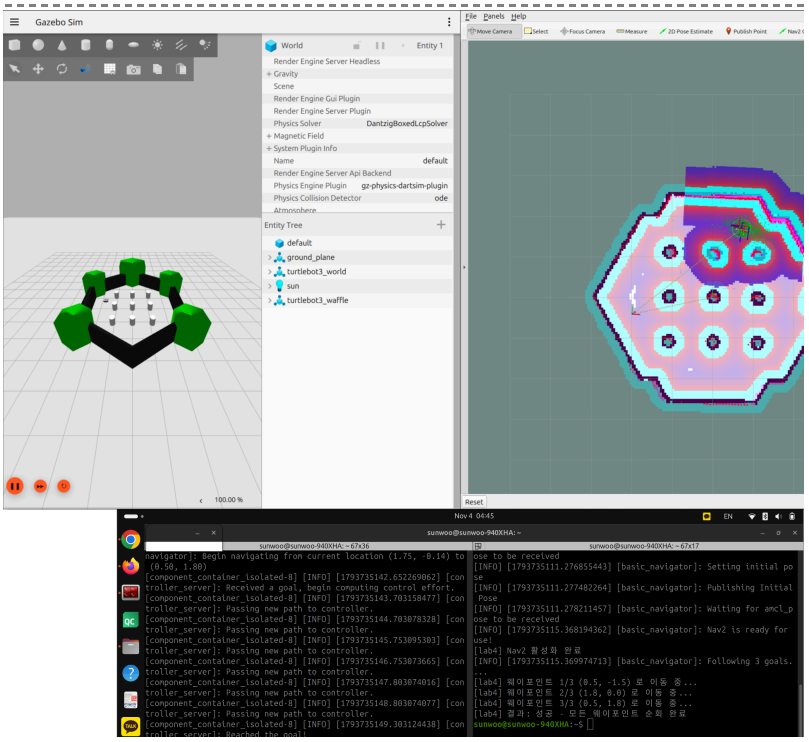


그림 9. lab4_commander.py로 웨이포인트 3개를 순회한 결과

스크립트를 실행하자 초기 위치가 자동으로 설정되었고, Nav2가 활성화된 뒤 로봇이 웨이포인트 1, 2, 3을 차례로 방문하였다. 터미널에는 다음과 같이 진행 상황과 결과가 출력되었다.

```
[lab4] 초기 위치 설정: (-2.0, -0.5, 0.0)
[lab4] Nav2 활성화 완료
[lab4] 웨이포인트 1/3 (0.5, -1.5) 로 이동 중...
[lab4] 웨이포인트 2/3 (1.8, 0.0) 로 이동 중...
[lab4] 웨이포인트 3/3 (0.5, 1.8) 로 이동 중...
[lab4] 결과: 성공 - 모든 웨이포인트 순회 완료
```

고찰

- 자동화의 장점: RViz에서 매번 목표를 클릭하는 대신 코드로 경로를 정의하면, 같은 주행을 반복 실험하거나 순찰·배송처럼 정해진 경로를 도는 작업에 바로 활용할 수 있다.
- yaw의 의미: 웨이포인트에 방향을 함께 지정하면, 로봇이 도착 후 해당 방향으로 회전하여 정렬한다.

5. 실습 5: STVL (Spatio-Temporal Voxel Layer)

배경 이론

기본 장애물 레이어는 한 번 기록된 장애물을, 라이다가 그 자리를 다시 관측해 비어 있다고 확인할 때까지 유지한다. 따라서 사람처럼 움직이는 물체가 지나간 뒤에도 그 흔적이 남아, 실제로는 비어 있는 공간을 로봇이 계속 피해 가는 유령 장애물 문제가 생길 수 있다.

STVL은 장애물을 3차원 복셀로 저장하고, 각 복셀에 시간 정보를 함께 기록한다. 일정 시간 동안 다시 관측되지 않은 복셀은 자동으로 제거된다.

구분	기본 장애물 레이어	STVL
저장 방식	2D/3D 격자	3D 복셀 + 시간 정보
장애물 제거	레이저가 빈 공간을 관측(clearing)할 때만	clearing + 시간 경과(decay)
움직이는 물체	흔적이 남기 쉬움	decay 시간 후 자동 제거
주요 파라미터	-	voxel_decay, decay_model, voxel_size

실습 방법

STVL 패키지를 설치한 뒤 lab5 스크립트를 실행하고, RViz에서 Add → By topic → /local_costmap/voxel_grid (PointCloud2)를 추가하여 복셀을 표시하였다.

```
sudo apt install ros-jazzy-spatio-temporal-voxel-layer
~/nav2_edu_ws/scripts/lab5_stvl.sh
```

decay 관련 설정은 params/nav2_params_stvl.yaml의 local_costmap → stvl_layer에 있다.

```
stvl_layer:
  plugin: "spatio_temporal_voxel_layer/SpatioTemporalVoxelLayer"
  voxel_decay: 5.0     # 관측이 끊긴 복셀이 사라지기까지 시간(초)
  decay_model: 0       # 0=선형, 1=지수, -1=사라지지 않음
  voxel_size: 0.05
```

로봇 앞에 장애물을 놓아 복셀이 생기는 것을 확인한 뒤, 장애물을 제거하고 복셀이 사라지는 과정을 관찰하였다.

실습 결과

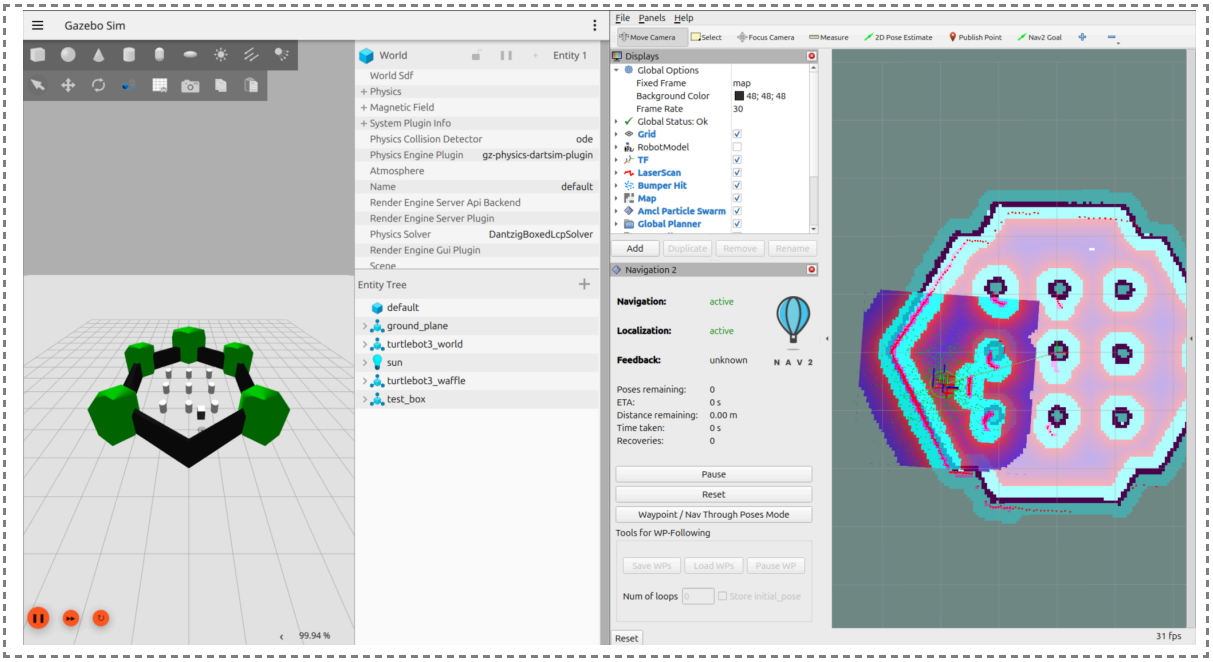


그림 10. 장애물을 놓았을 때 로봇 앞에 복셀이 생성된 모습

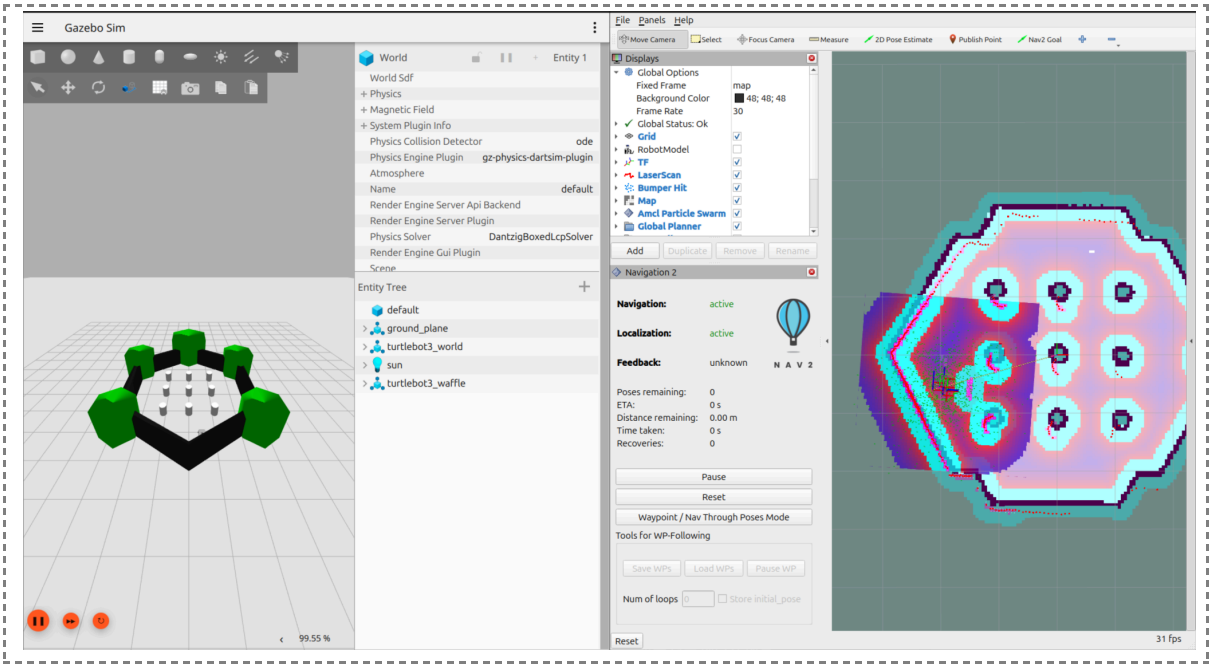


그림 11. 장애물을 제거한 뒤

장애물을 놓자 라이다가 이를 관측하여 로봇 앞에 복셀과 costmap 비용 영역이 생성되었다.

장애물을 제거한 뒤에는 복셀이 곧바로 사라지지 않고 남아 있다가, 일정 시간이 지난 후 사라졌다.

고찰

- voxel_decay의 trade-off: 값이 작으면 지나간 물체의 흔적이 빨리 사라져 유령 장애물이 줄어들지만, 잠시 시야에서 벗어난 실제 장애물도 금방 잊어버려 충돌 위험이 생긴다. 값이 크면 장애물을 오래 기억해 안전하지만 유령 장애물이 오래 남는다.
- clearing과 decay의 차이: 라이다가 장애물이 있던 자리를 계속 보고 있으면 비어 있다는 관측으로 decay 시간보다 빨리 지워질 수 있다. decay는 라이다가 보지 못하는 영역의 정보를 정리하는 역할을 한다.
- 활용: 사람이 많이 지나다니는 실내나 물류 창고처럼 동적 장애물이 많은 환경에서 STVL이 유용하며, 3D 센서(깊이 카메라, 3D 라이다)와 함께 쓰면 높이가 다른 장애물도 표현할 수 있다.